

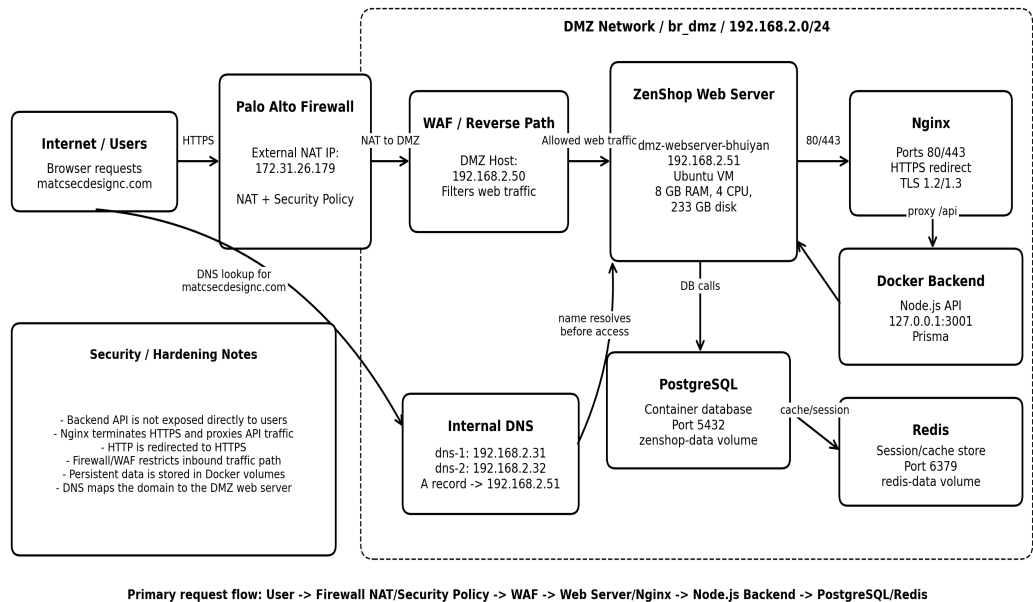
ZenShop E-Commerce: Build & Security Documentation

Installation Instructions | Configuration Documentation | Hardening Guidelines

Project	E-Commerce Website
Student	Jobayer Bhuiyan
Environment	Proxmox / NetLab DMZ web server
Primary Domain	matcsecdesignc.com / www.matcsecdesignc.com
Web Server IP	192.168.2.51

ZenShop Logical Network Diagram

E-commerce web service deployment for Security Design final documentation



1. Project Overview

This document serves as the comprehensive build and security reference for the ZenShop e-commerce platform, deployed within the Security Design NetLab environment. The objective was to deploy a fully functional e-commerce application into a DMZ, securely connect a React frontend to a Node.js backend and PostgreSQL database, publish the service via Nginx with HTTPS, and implement strict security controls to protect the infrastructure.

1.1 Completed Architecture & Services

- **DMZ Web Server:** Ubuntu-based server configured with static addressing on the `br_dmz` network.
- **Application Deployment:** ZenShop source code cloned from GitHub and hosted locally.

- **Containerization (Docker Compose):** Isolated services for the Node.js backend, PostgreSQL database, and Redis cache.
- **Database Management:** Prisma ORM utilized for successful schema migration and data seeding; product inventory is dynamically rendered on the website.
- **Reverse Proxy:** Nginx configured to serve static frontend assets and securely proxy API traffic to the internal backend.
- **Encryption:** HTTPS enforced using a locally generated TLS certificate, with automatic HTTP-to-HTTPS redirection.
- **Networking & Access:** DNS and firewall policies strictly configured so `matcsecdesignc.com` routes exclusively to the ZenShop web service.

1.2 Server Resources

Component	Allocation
Memory	8 GB
Processor	4 vCPU (1 socket, 4 cores)
Machine Type	i440fx
Hard Disk	233 GB
Network Bridge	br_dmz

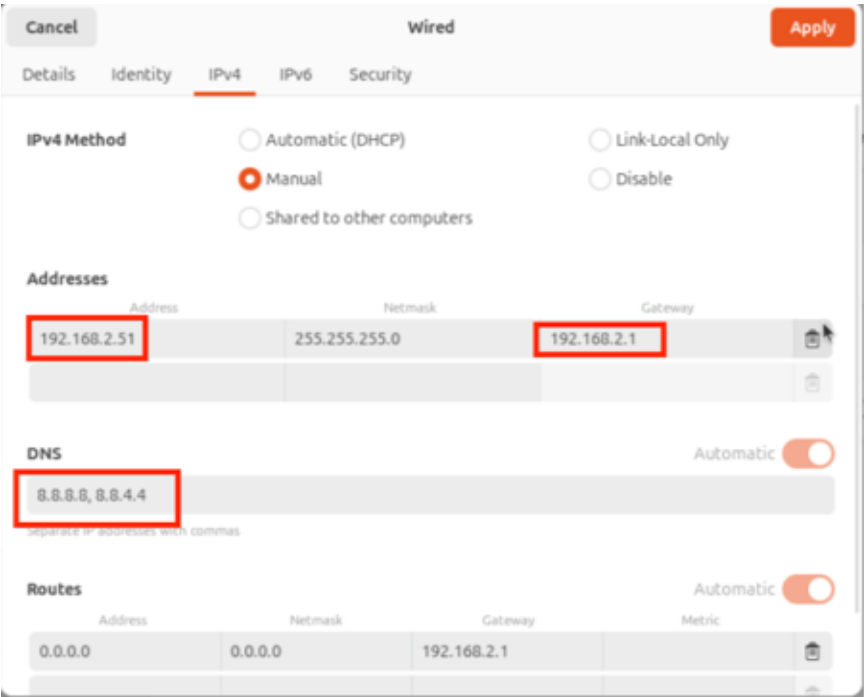


Figure 1. Static IPv4 configuration for the DMZ web server: IP `192.168.2.51/24` , Gateway `192.168.2.1` , and DNS `8.8.8.8 / 8.8.4.4` .

2. Detailed Step-by-Step Installation Instructions

This section outlines the baseline installation, starting from a fresh DMZ web server to a containerized, functioning application stack.

2.1 Configure the Web Server Network

To ensure the server is reliably reachable within the DMZ and can be mapped to DNS, a static IP is required.

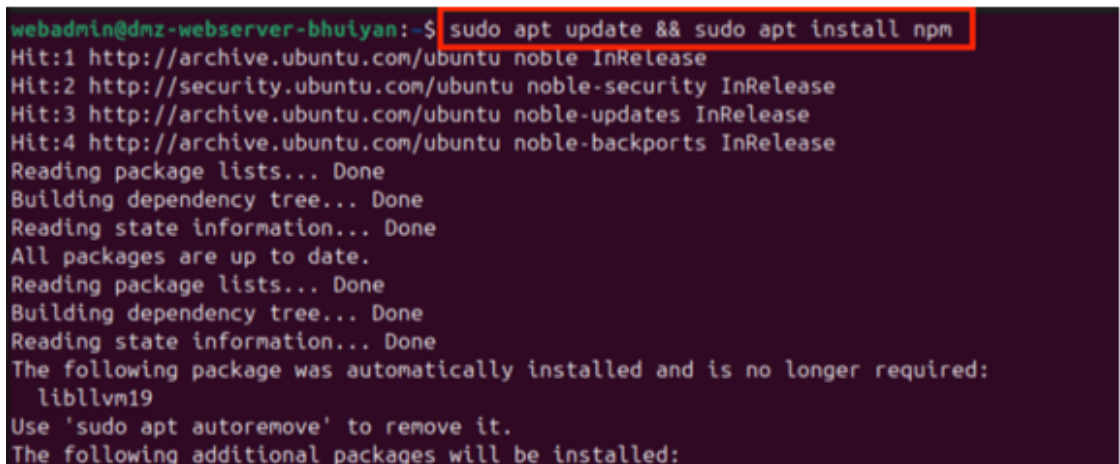
1. Open the wired IPv4 settings on the Ubuntu DMZ web server.
2. Set the IPv4 method to **Manual**.
3. Assign the IP address `192.168.2.51` with the subnet mask `255.255.255.0`.
4. Set the gateway to `192.168.2.1`.
5. Set DNS to `8.8.8.8` and `8.8.4.4` to allow for initial package installation and system updates.
6. Apply the network settings and verify external connectivity:

```
ping 8.8.8.8
```

2.2 Install Required System Packages

Update the Ubuntu package indexes and install the foundational tools required for the build.

```
sudo apt update
sudo apt install npm git -y
```



```
webadmin@dmz-webserver-bhuiyan:~$ sudo apt update && sudo apt install npm
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
```

Figure 2. Updating system packages and installing the Node Package Manager (npm).

```
webadmin@dmz-webserver-bhuiyan:~$ sudo apt install git
[sudo] password for webadmin:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
webadmin@dmz-webserver-bhuiyan:~$
```

Figure 3. Git successfully installed on the web server.

2.3 Clone the ZenShop Repository

Retrieve the application source code from the repository.

```
git clone https://github.com/jbhuiyan-matc/ZenShop.git
cd ZenShop
```

```
webadmin@dmz-webserver-bhuiyan:~$ git clone https://github.com/jbhuiyan-matc/ZenShop.git
Cloning into 'ZenShop'...
remote: Enumerating objects: 16226, done.
remote: Counting objects: 100% (385/385), done.
remote: Compressing objects: 100% (195/195), done.
remote: Total 16226 (delta 175), reused 380 (delta 174), pack-reused 15841 (from 1)
Receiving objects: 100% (16226/16226), 31.45 MiB | 20.75 MiB/s, done.
Resolving deltas: 100% (4511/4511), done.
webadmin@dmz-webserver-bhuiyan:~$
```

Figure 4. ZenShop repository cloned to the local DMZ server.

2.4 Prepare the Frontend Environment

The frontend is a React application built with Vite. We first install its dependencies to verify it runs, though product data requires the backend to function.

```
cd ZenShop/frontend
npm install
npm run dev
```

```
webadmin@dmz-webserver-bhuiyan:~$ cd ZenShop/frontend/
```

Figure 5. Navigating into the frontend directory.

```
webadmin@dmz-webserver-bhuiyan:~/ZenShop/frontend$ sudo npm run dev

> zenshop-frontend@0.1.0 dev
> vite

VITE v5.4.21 ready in 186 ms
→ Local: http://localhost:3000/
→ Network: use --host to expose
→ press h + enter to show help
```

Figure 6. Running the Vite frontend development server on `localhost:3000`.

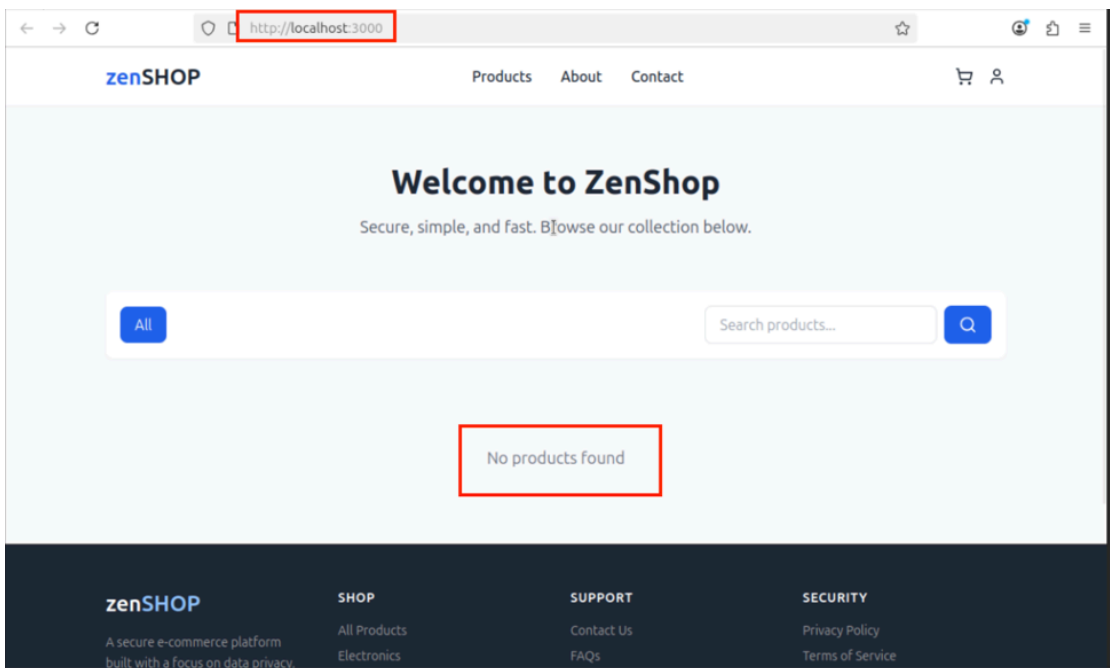


Figure 7. The frontend page loads successfully, but no products appear because the backend API is not yet running. This proves the UI is functional but awaiting data.

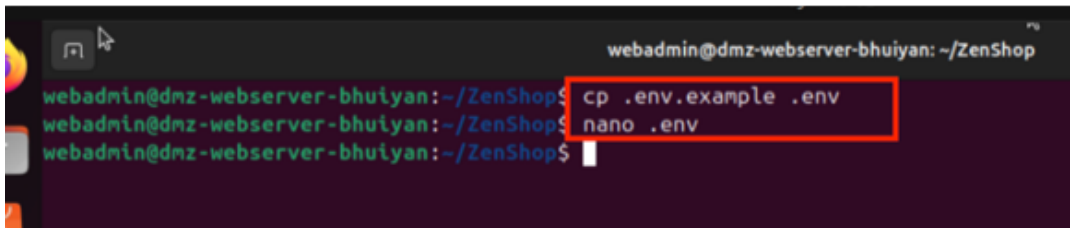
2.5 Create the Backend Environment Configuration

Environment variables securely inject database credentials and cryptographic secrets without hardcoding them into the source code.

```
cd ../backend
cp .env.example .env
nano .env
```

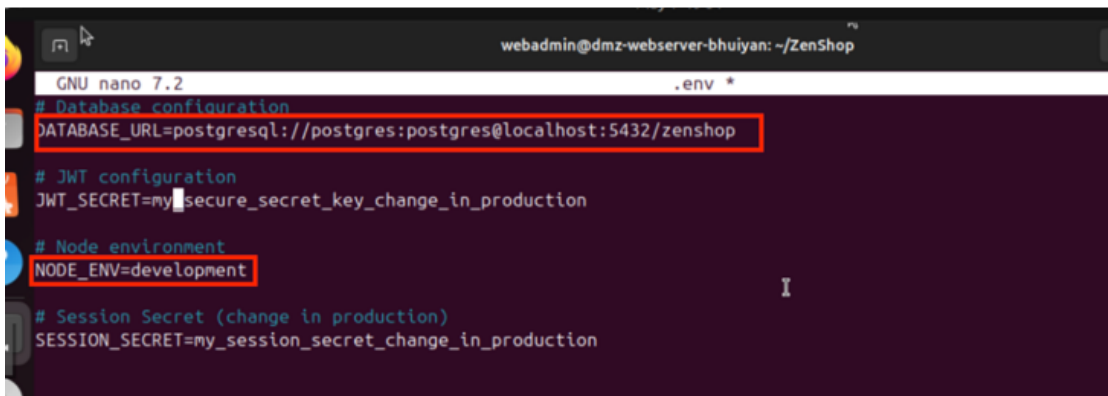
Ensure the file contains the necessary database connection string and strongly generated secrets for JWT (JSON Web Tokens) and session management:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/zenshop
NODE_ENV=production
JWT_SECRET=your_secure_jwt_secret_key_here
SESSION_SECRET=your_secure_session_secret_key_here
```



```
webadmin@dmz-webserver-bhuiyan: ~/ZenShop
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ cp .env.example .env
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ nano .env
webadmin@dmz-webserver-bhuiyan:~/ZenShop$
```

Figure 8. Copying `.env.example` to `.env` and opening the environment configuration file.



```
GNU nano 7.2 .env *
# Database configuration
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/zenshop

# JWT configuration
JWT_SECRET=my_secure_secret_key_change_in_production

# Node environment
NODE_ENV=development

# Session Secret (change in production)
SESSION_SECRET=my_session_secret_change_in_production
```

Figure 9. The configured backend `.env` file showing the database connection string and environment variables.

2.6 Install Docker and Docker Compose

Docker is utilized to isolate the backend, database, and caching services, ensuring consistent environments and easier management.

```
sudo apt install -y docker.io docker-compose-v2
sudo systemctl start docker
sudo systemctl enable docker
```

```

webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo apt install -y docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  liblvm19
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  bridge-utils containerd pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-
  rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd docker.io pigz runc ubuntu-fan
0 upgraded, 6 newly installed, 0 to remove and 0 not upgraded.
Need to get 73.4 MB of archives.
After this operation, 278 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]

```

Figure 10. Installing the Docker Engine.

```

webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo apt install -y docker-compose-v2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  liblvm19
Use 'sudo apt autoremove' to remove it.
The following NEW packages will be installed:
  docker-compose-v2
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 14.8 MB of archives.
After this operation, 66.6 MB of additional disk space will be used.
Ign:1 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker-compose-v2 amd64 2.40.3+ds1-0ubuntu1-24.04.1
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker-compose-v2 amd64 2.40.3+ds1-0ubuntu1-24.04.1
[14.8 MB]
Fetched 14.8 MB in 27s (549 kB/s)
Selecting previously unselected package docker-compose-v2.
(Reading database ... 225952 files and directories currently installed.)
Preparing to unpack .../docker-compose-v2_2.40.3+ds1-0ubuntu1-24.04.1_amd64.deb ...
Unpacking docker-compose-v2 (2.40.3+ds1-0ubuntu1-24.04.1) ...
Setting up docker-compose-v2 (2.40.3+ds1-0ubuntu1-24.04.1) ...
webadmin@dmz-webserver-bhuiyan:~/ZenShop$

```

Figure 11. Installing Docker Compose v2.

```

webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo systemctl start docker
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo systemctl enable docker
webadmin@dmz-webserver-bhuiyan:~/ZenShop$

```

Figure 12. Starting and enabling the Docker service to ensure containers launch automatically upon server reboot.

2.7 Start the Application Stack

We use Docker Compose to build and spin up the complete backend infrastructure. This provisions persistent volumes for PostgreSQL (user/product data) and Redis (session/caching data).

```

cd ~/ZenShop
docker compose version
sudo docker compose up -d --build

```

```
webadmin@dmz-webserver-bhuiyan: ~/ZenShop
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo docker compose up -d --build
[*] Running 28/28
✓ zenshop-redis Pulled
  897d797d2723 Pull complete 3.5s
  6447bc944d85 Pull complete 0.8s
  51bf34b872a6 Pull complete 0.2s
  1f0676652ae4 Pull complete 0.9s
  9572acab3f32 Pull complete 0.3s
  2da9501c87e4 Pull complete 0.4s
  4f4fb700ef54 Pull complete 0.2s
  47836b8a3274 Pull complete 0.2s
  c3c3ab8d6b01 Download complete 1.3s
  2074d4e4f1bb Download complete 0.0s
✓ zenshop-postgres Pulled
  cef33b0ec231 Pull complete 5.0s
  a13375e48b67 Pull complete 0.3s
  9e42eccf4bc0 Pull complete 0.4s
  8e80c373fbad Pull complete 0.3s
  3531af2bc2a9 Pull complete 4.2s
  9ebf6f2e52f3 Pull complete 1.9s
  d65fa596db1b Pull complete 0.3s
  3326b6522e5b Pull complete 0.3s
  3618b5f7b5bc Pull complete 0.3s
  decd178ddeb0 Pull complete 2.1s
  f2aef946c609 Pull complete 0.3s
  102496efe6c1 Pull complete 2.3s
  a88738cb3441 Pull complete 0.3s
  2952c21578f5 Pull complete 0.7s
  72a8a4ef0d8f Download complete 0.0s
  61438927987f Download complete 0.2s
WARNING! Docker Compose is configured to build using Bake, but buildx isn't installed
```

Figure 13. Docker Compose building the backend application image and pulling required database containers.

```
✓ zenshop-backend Built 0.0s
✓ Network zenshop_default Created 0.0s
✓ Volume zenshop_zenshop-data Created 0.0s
✓ Volume zenshop_zenshop-redis-data Created 0.0s
✓ Container zenshop-zenshop-redis-1 Healthy 5.9s
✓ Container zenshop-zenshop-postgres-1 Healthy 5.9s
✓ Container zenshop-zenshop-backend-1 Started 5.8s
webadmin@dmz-webserver-bhuiyan:~/ZenShop$
```

Figure 14. Backend, PostgreSQL, and Redis containers successfully created and started in detached mode.

2.8 Generate Prisma Client, Apply Migrations, and Seed Data

With the database running, we must apply our schema and seed initial inventory data.

```
sudo docker compose exec zenshop-backend npx prisma generate
sudo docker compose exec zenshop-backend npx prisma migrate deploy
sudo docker compose exec zenshop-backend npm run seed
```



```
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo docker compose exec zenshop-backend npx prisma generate
Prisma schema loaded from prisma/schema.prisma

✓ Generated Prisma Client (v5.22.0) to ./node_modules/@prisma/client in 82ms

Start by importing your Prisma Client (See: https://pris.ly/d/importing-client)

Tip: Curious about the SQL queries Prisma ORM generates? Optimize helps you enhance your visibility: https://pris.ly/tip-2-optimize

Update available 5.22.0 -> 7.8.0

This is a major update - please follow the guide at https://pris.ly/d/major-version-upgrade

Run the following to update
  npm i --save-dev prisma@latest
  npm i @prisma/client@latest

webadmin@dmz-webserver-bhuiyan:~/ZenShop$
```

Figure 15. The Prisma client successfully generated inside the backend container.

```
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo docker compose exec zenshop-backend npx prisma migrate deploy
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "zenshop", schema "public" at "zenshop-postgres:5432"

1 migration found in prisma/migrations

No pending migrations to apply.
webadmin@dmz-webserver-bhuiyan:~/ZenShop$
```

Figure 16. Prisma migrations successfully deployed against the internal `zenshop-postgres:5432` database.

```
webadmin@dmz-webserver-bhuiyan:~/ZenShop$ sudo docker compose exec zenshop-backend npm run seed

> zenshop-backend@0.1.0 seed
> node prisma/seed.js

Seeding database...
Admin user created: admin@zenshop.com
Categories created: Electronics, Clothing, Home & Garden, Sports & Outdoors
Products created: 16
Database seeding completed successfully!
webadmin@dmz-webserver-bhuiyan:~/ZenShop$
```

Figure 17. Database seed completed; initial product categories, 16 sample products, and the default administrator account were successfully created.

2.9 Build and Publish the Frontend

Prepare the frontend for production by compiling the assets and moving them to the Nginx web root.

```
cd ~/ZenShop/frontend
npm run build
sudo mkdir -p /var/www/zenshop
sudo cp -r dist/* /var/www/zenshop/
```

```
webadmin@dmz-webserver-bhuiyan:~/ZenShop/frontend$ npm run build

> zenshop-frontend@0.1.0 build
> vite build

vite v5.4.21 building for production...
✓ 1382 modules transformed.
dist/index.html                0.87 kB | gzip: 0.46 kB
dist/assets/index-BN4zp51L.css 26.55 kB | gzip: 5.34 kB
dist/assets/shield-hHRT53JN.js 0.15 kB | gzip: 0.16 kB
dist/assets/search-C1z0p_QT.js 0.17 kB | gzip: 0.16 kB
dist/assets/eye-Cyz63Rvl.js    0.20 kB | gzip: 0.18 kB
dist/assets/map-pin-DIIDPTag.js 0.20 kB | gzip: 0.19 kB
dist/assets/credit-card-Alm_a4gM.js 0.20 kB | gzip: 0.18 kB
dist/assets/lock-CuyYnNYd.js   0.20 kB | gzip: 0.19 kB
dist/assets/mail-DG_RXupm.js   0.21 kB | gzip: 0.19 kB
dist/assets/truck-BO3z8mYM.js  0.35 kB | gzip: 0.25 kB
dist/assets/NotFound-BAfoedaY.js 0.41 kB | gzip: 0.27 kB
```

Figure 18. The Vite frontend production build completed successfully.

```
webadmin@dmz-webserver-bhuiyan:~/ZenShop/frontend$ sudo mkdir -p /var/www/zenshop
webadmin@dmz-webserver-bhuiyan:~/ZenShop/frontend$ sudo cp -r dist/* /var/www/zenshop/
cp: cannot stat 'dist/*': No such file or directory
webadmin@dmz-webserver-bhuiyan:~/ZenShop/frontend$ sudo cp -r dist/* /var/www/zenshop/
webadmin@dmz-webserver-bhuiyan:~/ZenShop/frontend$
```

Figure 19. Creating the `/var/www/zenshop` directory and copying the compiled static files into the Nginx web root.

3. Detailed Step-by-Step Configuration Documentation

3.1 Verify Application Functionality Before Nginx

Before introducing Nginx into the architecture, we verify that the frontend and backend communicate successfully.

```
cd ~/ZenShop/frontend
npm run dev
```

```
webadmin@dmz-webserver-bhuiyan: ~/ZenShop/frontend$ sudo npm run dev
[sudo] password for webadmin:

> zenshop-frontend@0.1.0 dev
> vite

VITE v5.4.21 ready in 183 ms

→ Local: http://localhost:3000/
→ Network: use --host to expose
→ press h + enter to show help
```

Figure 20. Running the frontend development server temporarily to test API connectivity.

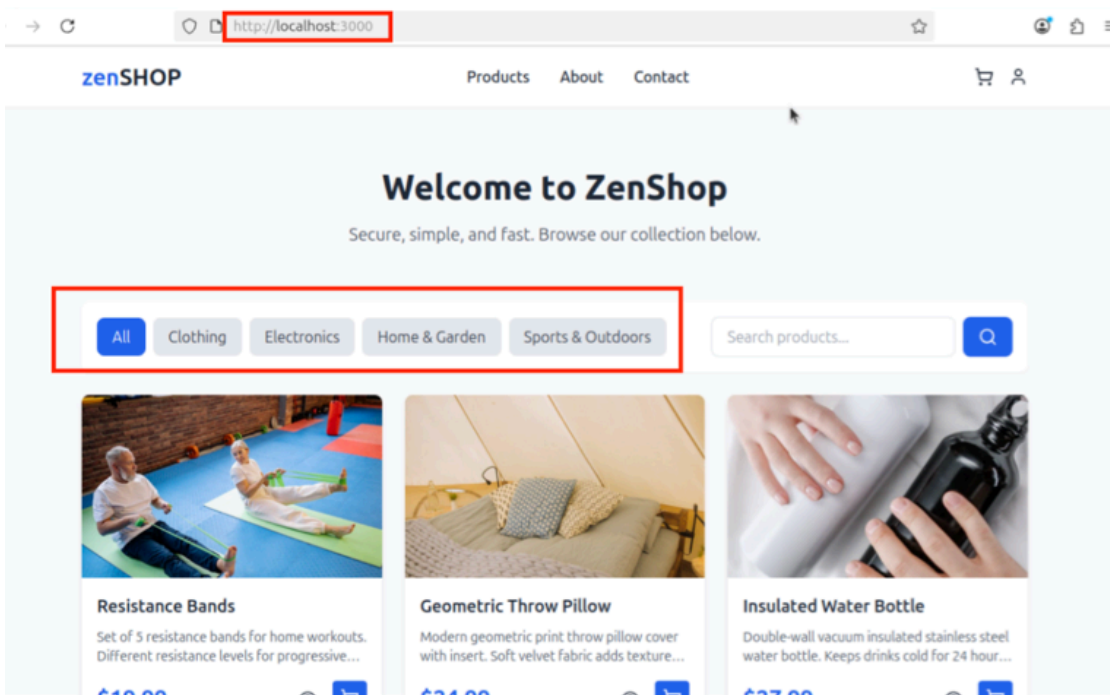


Figure 21. Product categories and inventory items are now fully visible. This proves the frontend is successfully retrieving data from the backend database via the API.

```
webadmin@dmz-webserver-bhuiyan: ~/ZenShop/backend$ sudo docker ps
[sudo] password for webadmin:
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
f479f5e262d8   zenshop-zenshop-backend            "docker-entrypoint.s..." 2 days ago    Up About a minute (healthy)    0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp, 5000/tcp
cf8fb3347693   postgres:16                        "docker-entrypoint.s..." 2 days ago    Up About a minute (healthy)    5432/tcp
a53aed3970bc   redis:7-alpine                     "docker-entrypoint.s..." 2 days ago    Up About a minute (healthy)    6379/tcp
```

Figure 22. Running `docker ps` proves the Backend, PostgreSQL, and Redis containers are running stably on their respective internal ports.

3.2 Install and Enable Nginx

Nginx acts as our primary entry point, serving the frontend files and reverse proxying API traffic.

```
sudo apt install -y nginx
sudo mkdir -p /etc/nginx/sites-available/ /etc/nginx/sites-enabled/
```

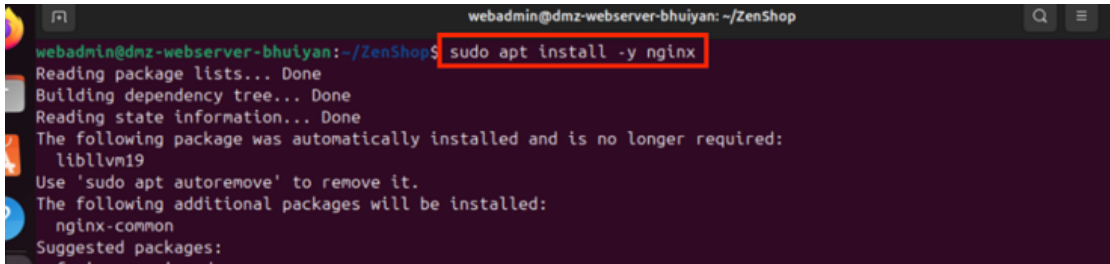
A terminal window with a dark background. The prompt is 'webadmin@dmz-webserver-bhuiyan: ~/ZenShop'. The command 'sudo apt install -y nginx' is entered and highlighted with a red box. The output shows the package lists being read, the dependency tree being built, and state information being read. It then states that the package was automatically installed and is no longer required, listing 'liblvm19' as a package that can be removed with 'sudo apt autoremove'. It also lists 'nginx-common' as an additional package to be installed and shows suggested packages.

Figure 23. Nginx successfully installed on the web server.

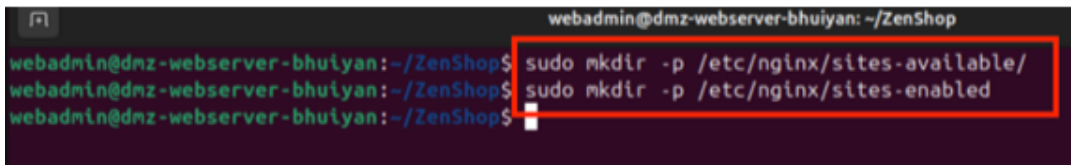
A terminal window with a dark background. The prompt is 'webadmin@dmz-webserver-bhuiyan: ~/ZenShop'. Two commands are entered and highlighted with a red box: 'sudo mkdir -p /etc/nginx/sites-available/' and 'sudo mkdir -p /etc/nginx/sites-enabled/'.

Figure 24. Ensuring the Nginx site configuration directories exist.

3.3 Configure the ZenShop Nginx Site

We copy our pre-configured Nginx file, enable it via a symbolic link, and remove the default Nginx site to prevent port conflicts.

```
sudo cp nginx/zenshop.conf /etc/nginx/sites-available/zenshop
sudo ln -s /etc/nginx/sites-available/zenshop /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
```

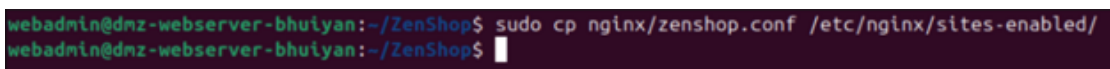
A terminal window with a dark background. The prompt is 'webadmin@dmz-webserver-bhuiyan: ~/ZenShop'. The command 'sudo cp nginx/zenshop.conf /etc/nginx/sites-enabled/' is entered and highlighted with a red box.

Figure 25. Copying the ZenShop Nginx configuration into `sites-available`.

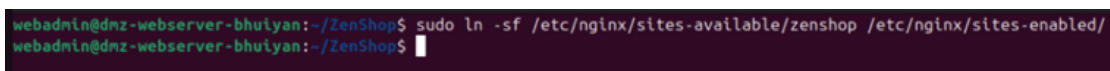
A terminal window with a dark background. The prompt is 'webadmin@dmz-webserver-bhuiyan: ~/ZenShop'. The command 'sudo ln -sf /etc/nginx/sites-available/zenshop /etc/nginx/sites-enabled/' is entered and highlighted with a red box.

Figure 26. Enabling the site by creating a symbolic link in `sites-enabled`.

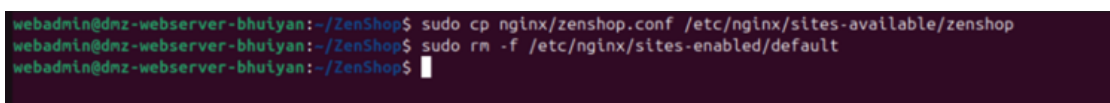
A terminal window with a dark background. The prompt is 'webadmin@dmz-webserver-bhuiyan: ~/ZenShop'. The command 'sudo rm -f /etc/nginx/sites-enabled/default' is entered and highlighted with a red box.

Figure 27. Deleting the default Nginx configuration to ensure our site handles all traffic.

3.4 Create SSL Certificate Directories and Certificate

To enforce HTTPS, we generate a local TLS certificate for our domain.

```
sudo mkdir -p /etc/ssl/certs /etc/ssl/private

sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/zenshop.key \
  -out /etc/ssl/certs/zenshop-fullchain.crt \
  -subj "/C=US/ST=WI/L=Madison/O=MATC/CN=matcsecdesignc.com"
```

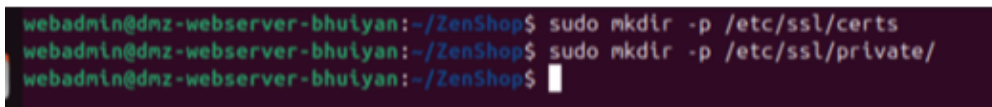
A terminal window showing the execution of two mkdir commands to create the directories /etc/ssl/certs and /etc/ssl/private. The prompt is webadmin@dmz-webserver-bhuiyan:~/ZenShop\$.

Figure 28. Creating secure directories to hold the cryptographic keys.

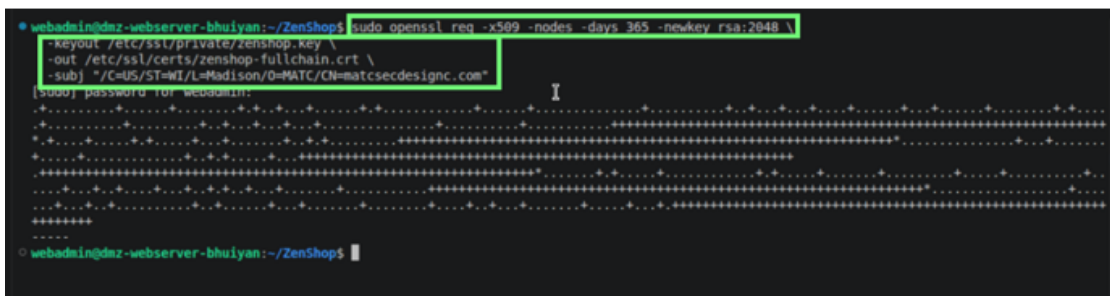
A terminal window showing the execution of the openssl req command to generate a self-signed X.509 certificate. The command is highlighted with a green box. The output shows the certificate details and a confirmation message. The prompt is webadmin@dmz-webserver-bhuiyan:~/ZenShop\$.

Figure 29. A self-signed X.509 certificate successfully generated for `matcsecdesignc.com`.

3.5 Test and Reload Nginx

```
sudo nginx -t
sudo systemctl reload nginx
```

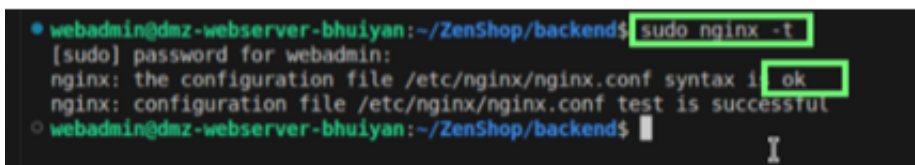
A terminal window showing the execution of the nginx -t command to test the configuration syntax. The command is highlighted with a green box. The output shows that the configuration file /etc/nginx/nginx.conf syntax is ok and the test is successful. The prompt is webadmin@dmz-webserver-bhuiyan:~/ZenShop/backend\$.

Figure 30. Nginx configuration syntax test passed, confirming our deployment is ready to accept traffic.

3.6 Nginx Reverse Proxy Configuration Breakdown

This configuration establishes three core security controls:

1. **HTTP to HTTPS Redirection:** Forces all traffic to be encrypted.

2. **Topology Hiding:** Clients interact with Nginx on port 443 , while API requests are securely forwarded to the Node.js backend on 127.0.0.1:3001 . The backend is never exposed directly.
3. **Modern Cryptography:** Enforces TLSv1.2 and TLSv1.3 while dropping insecure legacy protocols.

```
# ZenShop Nginx Configuration
server {
    listen 80;
    server_name matcsecdesignc.com www.matcsecdesignc.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name matcsecdesignc.com www.matcsecdesignc.com;

    ssl_certificate /etc/ssl/certs/zenshop-fullchain.crt;
    ssl_certificate_key /etc/ssl/private/zenshop.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-
ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 1d;

    root /var/www/zenshop;
    index index.html;

    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml
application/xml text/javascript;
    gzip_min_length 1000;

    # Reverse Proxy for API Traffic
    location /api {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        proxy_connect_timeout 10s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    location /health {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
```

```
    proxy_set_header X-Real-IP $remote_addr;
}

location / {
    try_files $uri $uri/ /index.html;
}

location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
}
```

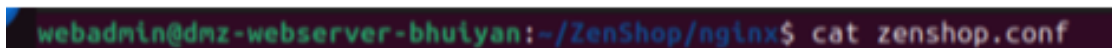


Figure 31. Displaying the contents of the deployed `zenshop.conf` file.

3.7 Configure Local Hostname Resolution

For isolated lab testing, the local `hosts` file is updated to route domain queries directly to the DMZ web server.

```
sudo nano /etc/hosts
```

Add the following mappings:

```
192.168.2.51 matcsecdesignc.com www.matcsecdesignc.com
192.168.2.51 store.matcsecdesignc.com
```

```
webadmin@dmz-webserver-bhuiyan: ~  
GNU nano 7.2 /etc/hosts  
127.0.0.1 localhost  
127.0.1.1 dmz-webserver  
  
192.168.2.51 admin.matcsecdesignnc.com  
192.168.2.51 store.matcsecdesignnc.com  
  
# The following lines are desirable for IPv6 capable hosts  
::1 ip6-localhost ip6-loopback  
fe00::0 ip6-localnet  
ff00::0 ip6-mcastprefix  
ff02::1 ip6-allnodes  
ff02::2 ip6-allrouters  
  
192.168.2.51 matcsecdesignnc.com www.matcsecdesignnc.com  
192.168.4.10 auth.matcsecdesignnc.com  
  
[ Bad lock file is ignored: /etc/.hosts.swp ]  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Figure 32. `/etc/hosts` entries proving localized domain resolution for the test environment.

3.8 Configure Firewall NAT and Security Policy

The perimeter firewall requires rules to permit and translate traffic bound for the application.

- **NAT Policy:** Translates external requests seamlessly to the internal DMZ web server IP (192.168.2.51).
- **Security Policy:** Explicitly permits web traffic (HTTP/HTTPS) to reach the service, blocking all unapproved ports.
- **Note for Presentation:** The backend API and databases are *never* exposed directly through the firewall; they remain safely behind Nginx inside the DMZ.

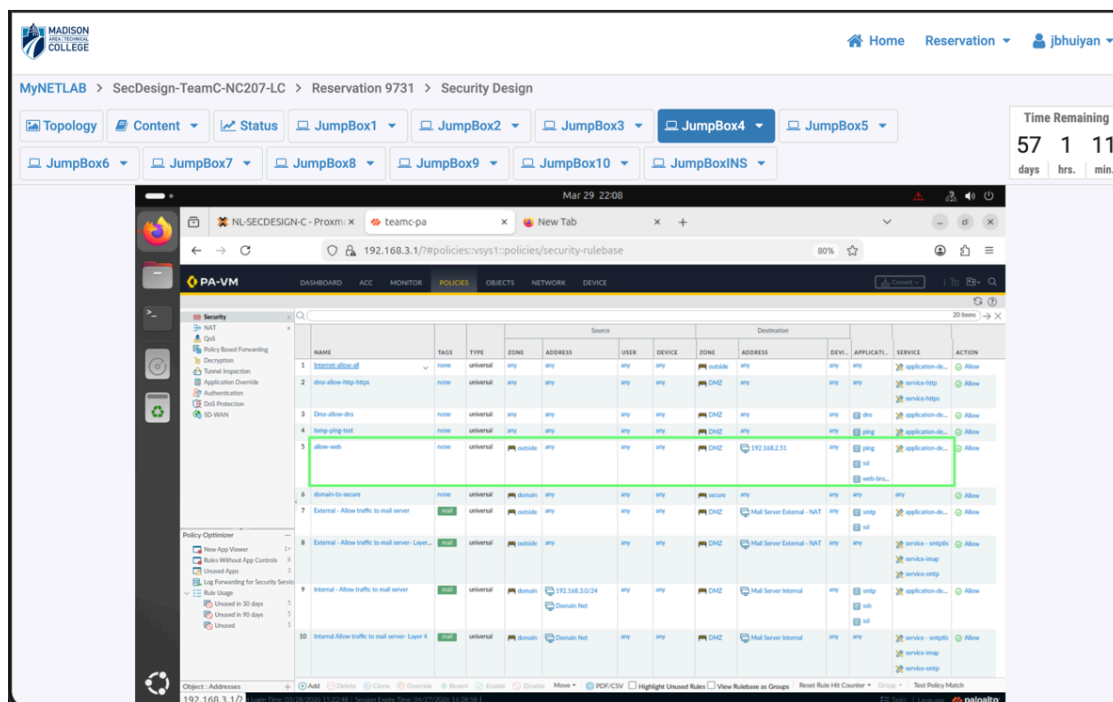


Figure 33. The Firewall NAT policy actively translating traffic toward the internal web server.

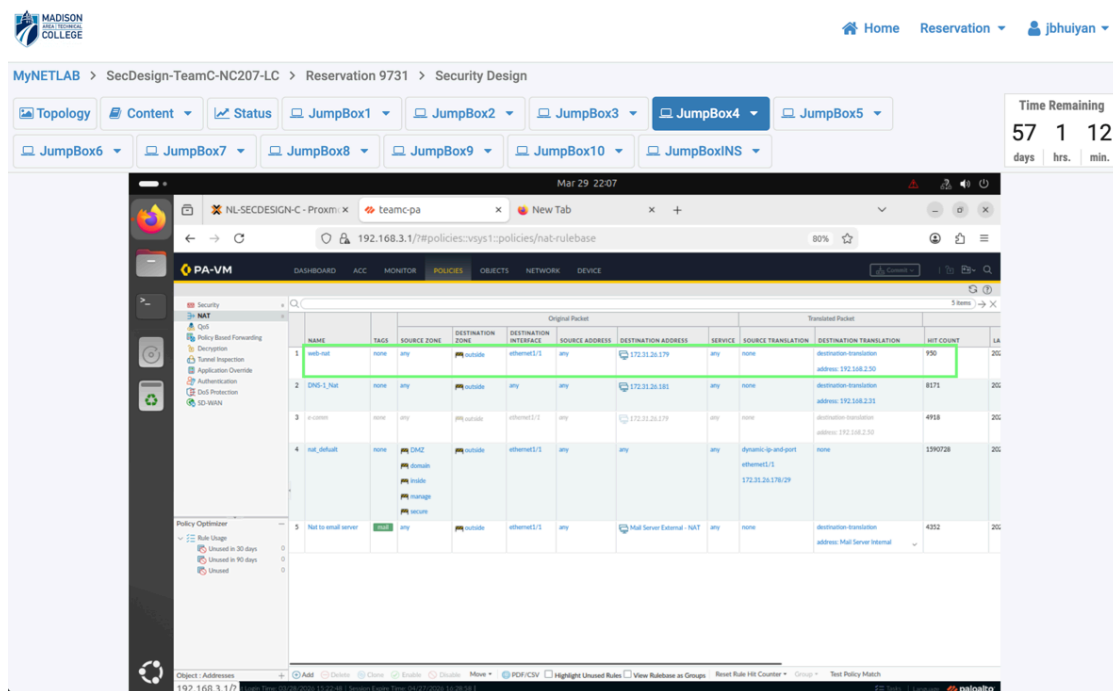


Figure 34. The Firewall Security policy actively enforcing access controls and allowing only approved WFA(Web Application Firewall) traffic.

3.9 Configure DNS Records

Figure 36. Final Validation: Navigating to `matcsecdesignnc.com` over HTTPS successfully loads the complete, interactive ZenShop storefront.

3.10 Validation Checklist

To confirm mission success, all core components were verified using the steps below:

Validation Item	Command / Action	Expected Result
Network Connectivity	<code>ping 8.8.8.8</code>	Server has reliable external network connectivity.
Docker Service	<code>sudo systemctl status docker</code>	Docker daemon is active and enabled for persistence.
Container Health	<code>sudo docker ps</code>	Backend, PostgreSQL, and Redis containers are running.
Database Status	<code>npx prisma migrate deploy</code>	Schema successfully deployed with no pending tasks.
Data Integrity	<code>npm run seed</code>	16 products and categories exist in the database.
Nginx Routing	<code>sudo nginx -t</code>	Nginx configuration test passes with zero errors.
Application Access	Browse to <code>https://matcsecdesignnc.com</code>	ZenShop loads securely, rendering product data.
Admin Authentication	Login with <code>admin@zenshop.com</code>	Credentials authenticate and grant admin privileges.

4. Detailed Hardening Guidelines Documentation

A "security-first" posture requires continuous application of defense-in-depth principles. The following hardening controls have been evaluated, applied, and documented for the ZenShop environment to defend against common attack vectors.

4.1 Infrastructure & Networking Defenses

Control Area	Hardening Action Implemented
Operating System	Maintain Ubuntu patch levels via <code>apt update</code> . Purge unused packages. Enforce non-root administrator accounts and restrict <code>sudo</code> privileges.
Firewall Exposure	Employ strict firewall rules. Expose only HTTP/HTTPS (ports 80/443). <i>Never</i> expose PostgreSQL, Redis, or Node.js directly to the internet.
WAF Integration	(Presentation Goal): Ensure the application rests behind a Web Application Firewall (WAF) to perform deep packet inspection of payloads.
Topology Isolation	Bind the Node.js backend to <code>127.0.0.1:3001</code> . This limits access exclusively to Nginx, dramatically shrinking the direct attack surface.

Database Protection	Isolate PostgreSQL within a Docker bridge network. Secure volumes with restricted permissions to prevent unauthorized host-level data access.
Cache Security	Confine Redis strictly to internal networks. External exposure of Redis could lead to session hijacking and cache poisoning.

4.2 Application & Cryptographic Defenses

Control Area	Hardening Action Implemented
Nginx Reverse Proxy	Position Nginx as the primary public edge. It sanitizes requests and only permits traffic to explicitly defined <code>/api</code> endpoints.
HTTPS Enforcement	Implement a mandatory 301 redirect on Port 80 to force all user sessions, credentials, and order data over encrypted Port 443 channels.
TLS Hardening	Disable legacy SSL protocols (SSLv3, TLS 1.0, TLS 1.1). Enforce TLSv1.2 and TLSv1.3 utilizing robust ECDHE/GCM cipher suites.
Certificate Management	Replace local lab certificates with trusted CA-signed certificates (e.g., Let's Encrypt) prior to production deployment. Automate renewals.
Secrets Management	Explicitly ignore <code>.env</code> files via <code>.gitignore</code> . Never commit database credentials, JWT keys, or API secrets to version control repositories.
JWT Session Integrity	Sign JSON Web Tokens with a highly entropic cryptographic secret. The backend drops any request containing a tampered or invalid JWT signature.
Rate Limiting	Apply aggressive rate-limiting thresholds to authentication routes to mitigate brute-force attacks, alongside standard limits for general API use.
Strict Input Validation	Enforce server-side schema validation (via the <code>zod</code> library) before data reaches PostgreSQL. Never blindly trust client-side frontend validation.
Security Headers (Helmet)	Inject protective HTTP response headers via <code>Helmet</code> middleware, actively defending against XSS, clickjacking, and MIME-sniffing vulnerabilities.
Credential Hashing	Passwords are one-way hashed and salted using Bcrypt . The database itself holds zero knowledge of a user's true plaintext password.
Least Privilege & Logging	Restrict database service accounts to necessary tables. Regularly audit <code>/var/log/nginx/access.log</code> and docker application logs for anomalies.

4.3 Ongoing Maintenance Routine

To ensure security longevity, administrators must adhere to the following checklist:

- ☐ Execute weekly system and dependency updates.
- ☐ Monitor container resource health and memory leaks using `sudo docker ps` and `docker compose logs`.
- ☐ Back up the persistent PostgreSQL Docker volume before modifying schemas.
- ☐ Rotate cryptographic secrets (`JWT_SECRET` , `SESSION_SECRET`) routinely.

- ☐ Continually verify that firewall routing safely passes through the designated WAF before striking the DMZ web server.
-

5. Final Summary

The ZenShop e-commerce platform has been successfully engineered, deployed, and secured within the local Proxmox / NetLab DMZ infrastructure.

The React frontend reliably serves end-users over the verified domain (`matcsecdesignnc.com`), seamlessly retrieving dynamic product data from the containerized Node.js backend and PostgreSQL database. Nginx functions precisely as designed: terminating HTTPS encryption, enforcing strong cryptography, and operating as a secure reverse proxy to shield internal services from the public perimeter.

By satisfying detailed installation methodologies, comprehensive configuration documentation, and an actionable hardening strategy, this project fully aligns with enterprise security design requirements.